



LogHound

A Custom Linux Security Log Analyzer

Project Report

Prepared by: Afsa Taj

Version 1.0.0

July 2026

Table of Contents

Table of Contents	2
1. Executive Summary	3
2. Project Objectives	3
3. Tool Identity.....	3
4. Key Features.....	3
4.1 Auth Log / SSH Analysis.....	3
4.2 Web Access Log Analysis	3
4.3 Reporting.....	4
5. How It Works	4
6. Installation.....	4
7. Running LogHound.....	5
8. Sample Findings Output.....	5
9. HTML Report Output	5
10. JSON Report Output	6
11. Testing & Validation.....	6
Detection Coverage Summary	6
12. Future Enhancements	6
13. Conclusion.....	7

1. Executive Summary

LogHound is a custom-built, lightweight command-line security tool designed to analyze Linux system logs (SSH/auth logs, syslog) and web server access logs (Apache/Nginx) to surface signs of malicious activity. It was built to give system administrators and security analysts a fast, dependency-free way to triage logs on a Linux box without needing a full SIEM deployment.

The tool automatically detects brute-force login attempts, account enumeration, privilege escalation, suspicious sudo usage, and common web application attack payloads such as SQL Injection, Cross-Site Scripting (XSS), and Path Traversal. Findings are scored by severity (Critical / High / Medium / Low) and can be exported as a console report, a JSON file for automation pipelines, or a styled HTML report for sharing with clients or management.

2. Project Objectives

- Build a single-file, dependency-free Python tool that runs on any standard Linux system.
- Detect the most common attacker behaviors seen in real-world SSH brute-force and web scanning activity.
- Provide clear, prioritized findings instead of raw, unfiltered log noise.
- Support multiple output formats (console, JSON, HTML) for different use cases — manual review, automation, and client reporting.
- Package the tool so it can be installed system-wide as a normal Linux CLI command.

3. Tool Identity

Name: LogHound

The name reflects the tool's purpose — like a hound tracking a scent, LogHound sniffs through large volumes of log data to track down suspicious activity that would otherwise be buried in noise. The name is original and not associated with any existing commercial product.

4. Key Features

4.1 Auth Log / SSH Analysis

- Brute-force detection — flags source IPs exceeding a configurable failed-login threshold.
- Account enumeration detection — flags repeated 'invalid user' probing from a single IP.
- Compromise indicator — flags a successful login from an IP that was just brute-forcing the host.
- Sensitive sudo command monitoring (passwd, useradd, visudo, chmod 777, etc.).
- Failed 'su' privilege escalation attempt detection.
- User/group creation and deletion event tracking.

4.2 Web Access Log Analysis

- SQL Injection payload detection.

- Cross-Site Scripting (XSS) payload detection.
- Path traversal / LFI-RFI / command injection detection.
- Known scanner tool signature detection (sqlmap, nikto, nmap, dirbuster, wpscan, etc.).
- Sensitive path probing detection (wp-login.php, .env, .git/, phpMyAdmin, etc.).
- Directory brute-force indicator via high-volume 404 tracking.

4.3 Reporting

- Color-coded console output for quick manual triage.
- JSON export for feeding into SIEMs or automation pipelines.
- Styled, shareable HTML report for client or management review.
- Top offending IP and top web client summaries.

5. How It Works

LogHound reads one or more log files line by line and runs each line through a set of purpose-built regular expressions tuned to common Linux auth log and web access log formats. Matches are aggregated in memory (grouped by IP, user, or attack category), then run through a correlation and scoring stage that converts raw counts into prioritized findings. The final stage renders the findings into the requested output format.

```
Log Files (auth/syslog/access)
|
v
Line-by-line Parser (regex engine)
|
v
Aggregation (per IP / user / category)
|
v
Correlation & Severity Scoring
|
v
Report Output (console / JSON / HTML)
```

6. Installation

LogHound requires only Python 3.6+ — no external libraries or pip installs are needed.

```
$ chmod +x loghound.py
$ sudo cp loghound.py /usr/local/bin/loghound
$ loghound -version
LogHound 1.0.0
```

```
(venv)-(root@kali)-[~/afsataj]
# git clone https://github.com/dendazle3-sketch/LogHound.git
Cloning into 'LogHound'...
remote: Enumerating objects: 11, done.
remote: Counting objects: 100% (11/11), done.
remote: Compressing objects: 100% (9/9), done.
remote: Total 11 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (11/11), 22.80 KiB | 1.08 MiB/s, done.

(venv)-(root@kali)-[~/afsataj]
# cd LogHound

(venv)-(root@kali)-[~/afsataj/LogHound]
# ls
LogHound  README.md

(venv)-(root@kali)-[~/afsataj/LogHound]
# cd LogHound

(venv)-(root@kali)-[~/afsataj/LogHound/LogHound]
# ls
LogHound_Project_Report.docx  loghound.py  USAGE.md
```

6. Running LogHound

Basic scan of an SSH auth log:

```
$ loghound -auth /var/log/auth.log
```

Combined SSH + web access log scan with an HTML report saved to disk:

```
$ loghound -auth /var/log/auth.log \  
--access /var/log/nginx/access.log \  
--output ~/reports/scan -format all
```

```

[1] Էլիբ ուր խոսու՞ք ԶԻՅԵՅՈՒՆ: \ԼՅԷ\ԴՈՅ\ՅՈՒՄ\ԴՈՅ
[ Գ ԵԼԻՅՈՒՆ ԴՈՅՈՒՍՈՒԿ.ԵԼ --ՅՈՒՄ \ԼՅԷ\ԴՈՅ\ՅՈՒՄ\ԴՈՅ --ԳՇԵԶԶ \ԼՅԷ\ԴՈՅ\ՍԷՅՈՒՄ\ԳՇԵԶԶ
(ԼԵԱԼ)-(ԼՈՒԴ\ԿԴԴ)-[-\ԿԷԶԵԿ\]ԴՈՅՈՒՍՈՒԿ\ԴՈՅՈՒՍՈՒԿ]

ԿՈ ԶՆԻԶԵՅՈՒՆ ԶԵԿԻԿԻԿ ԳԵԼԵԿԵԳ՝

ԳԴԿԿ ԵՅԼԴՈՅԶԷ : 0
ՄԵՐ ԵԴՆԵԶԷԶ : 0
ԶՈՓ ՇՈՄՍՈՅԶԷ : 0
ԳՇԵԶԵԳ ԴՈՅԻՆ : 0
ԷՅԴԵԳ ԴՈՅԻՆ : 0
ԷԼԵԶ ԵՐՇԵԶԶԵԳ : 0
ԴԻՅԶ ԵՐՇԵԶԶԵԳ : 0

== ՇՈՄՈՒԿ ==

ԴԻԵԶ ԵՐՇԵԶԶԵԳ : 0
ԷԼԵԶ ՍՈՅԼՅԵԳ : 0

[1] Էլիբ ուր խոսու՞ք ԶԻՅԵՅՈՒՆ: \ԼՅԷ\ԴՈՅ\ՅՈՒՄ\ԴՈՅ
[ Գ ԵԼԻՅՈՒՆ ԴՈՅՈՒՍՈՒԿ.ԵԼ --ՅՈՒՄ \ԼՅԷ\ԴՈՅ\ՅՈՒՄ\ԴՈՅ --ԳՇԵԶԶ \ԼՅԷ\ԴՈՅ\ՍԷՅՈՒՄ\ԳՇԵԶԶ
(ԼԵԱԼ)-(ԼՈՒԴ\ԿԴԴ)-[-\ԿԷԶԵԿ\]ԴՈՅՈՒՍՈՒԿ\ԴՈՅՈՒՍՈՒԿ]

--ԼԵԶԵՅՈՒ
--ՍՈ-ՇՈՐՈՒ
--ԽՈՄԿԻ [ՇՈՅՈՒԴԵՂ՝ՆՈՒՄԻՂ՝ԿԴԴ]
--ՈՒՐՈՒՐ՝ -ՇՈ ԵՄԻՆ ԵՅԶԵ ՈՒՐՈՒՐ ԵՄԻ (ԿԴԻՄՈՒՐ ԵՐԽԵԶԵՅՈՒՆ) ԽՈՒ ԵԼՈՒՐ ԷԼԵԶ

```

7. Sample Findings Output

Below is an example of the type of findings LogHound produces when run against a log containing a brute-force attack and a web attack attempt:

Severity	Category	Message
CRITICAL	Possible Compromise	Successful login from 45.33.12.10 (root) after 8 prior failures
HIGH	Brute Force	8 failed SSH password attempts from 45.33.12.10
HIGH	Web Attack: SQL Injection	1 SQL Injection attempt detected from 198.51.100.9
MEDIUM	Sensitive Path Probing	203.0.113.5 probed 3 sensitive paths (e.g. /wp-login.php)
MEDIUM	Account Change	User Added: backdoor_user

11. Testing & Validation

LogHound was validated against a set of crafted sample logs simulating: an SSH brute-force campaign from a single IP, subsequent account enumeration, a successful login immediately following the brute-force attempts (compromise indicator), risky sudo command usage, user account creation/deletion, and a web access log containing SQL Injection, XSS, and path traversal payloads from a scanner tool (sqlmap). All scenarios were correctly identified and scored by severity.

Detection Coverage Summary

Test Scenario	Detected?	Severity Assigned
SSH brute-force (8 attempts)	Yes	HIGH
Account enumeration (5 invalid users)	Yes	HIGH
Login success after brute-force	Yes	CRITICAL
Risky sudo command (passwd root)	Yes	MEDIUM
Failed su privilege escalation	Yes	MEDIUM
User account added/deleted	Yes	MEDIUM
SQL Injection payload	Yes	HIGH
XSS payload	Yes	HIGH
Path traversal payload	Yes	HIGH
Scanner tool signature (sqlmap)	Yes	MEDIUM
Sensitive path probing (wp-login.php etc.)	Yes	MEDIUM

12. Future Enhancements

- Real-time log tailing (monitor mode using inotify instead of one-shot scans).
- GeoIP lookups to enrich offending IPs with country/ASN data.
- Email/Slack/webhook alerting for CRITICAL findings.

- Configurable rules via an external YAML/JSON rules file instead of hard-coded regex.
- Support for additional log formats (Windows Event Logs, firewall logs, cloud provider logs).

13. Conclusion

LogHound successfully meets its design goals: a fast, dependency-free, single-file Linux log analyzer that surfaces high-value security findings from raw log noise. Its modular detection design makes it straightforward to extend with new attack signatures, and its multi-format output (console, JSON, HTML) makes it usable both for hands-on triage and for integration into broader security workflows.

— *End of Report* —